

LLM-Powered Query Monitoring and Optimization for Reproducible Big Data Workloads

Shouvik Sharma
Independent Researcher
India
Email: TODO

Abstract—Big data platforms execute heterogeneous analytical SQL workloads across scientific, operational, and security datasets. Inefficient queries can increase cost, delay downstream analytics, and reduce trust in shared data infrastructure. This paper presents a reproducible framework for LLM-powered query monitoring and optimization. The framework ingests public datasets, executes a controlled DuckDB workload, stores query telemetry in SQLite, analyzes SQL with a large language model, records optimization recommendations, and validates rewritten queries through execution-based semantic checks. In a pilot evaluation over three public datasets and eight queries, including four intentionally inefficient variants, the framework achieved 100% detection accuracy for the tested anti-patterns, zero false positives on baseline queries, a 75% semantic-match rate for rewritten queries, and total LLM API cost of \$0.000671. The evaluation uses 3-row pilot tables and should not be interpreted as a production-scale performance benchmark. Instead, the contribution is an auditable artifact and evaluation harness for studying foundation-model-assisted query governance in big data systems.

Index Terms—big data systems, query monitoring, SQL optimization, large language models, reproducibility, benchmark artifacts

I. INTRODUCTION

Big data systems are increasingly used to integrate scientific, environmental, cybersecurity, and operational datasets. As these platforms grow, SQL workloads become more diverse: analysts issue ad hoc queries, dashboards run recurring aggregations, and data applications generate programmatic SQL. Inefficient queries can silently increase runtime, waste compute, and produce avoidable operational cost. Query monitoring is therefore a governance problem as well as a performance problem.

Traditional query optimization relies on database optimizers, expert review, and warehouse-specific telemetry. These mechanisms remain essential, but they do not fully address the human-facing problem of explaining why a query is risky, identifying common anti-patterns, and proposing understandable rewrites. Recent foundation models create an opportunity to augment query review with natural-language reasoning and code transformation. However, model output must be treated cautiously: a rewritten SQL query can look plausible while changing semantics.

This paper presents an LLM-powered query monitoring framework designed for reproducible experimentation. The system downloads public datasets, constructs a controlled

workload, executes queries against DuckDB [1], [2], records telemetry in a SQLite query-history database [3], analyzes SQL with an LLM, stores recommendations, executes rewritten queries, and reports detection, semantic-match, runtime, and cost metrics. The framework is open source and available as an artifact [4].

The paper is positioned for IEEE BigData 2026 under Big Data Infrastructure, Big Data Management, Big Data Benchmarks, and Foundation Models for Big Data. It contributes: (1) a modular query-monitoring architecture for foundation-model-assisted SQL review; (2) a reproducible artifact that links public datasets, workload execution, query history, LLM analysis, and reporting; (3) a pilot benchmark measuring anti-pattern detection, rewrite validation, and LLM API cost; and (4) an explicit discussion of limitations needed before claiming production-scale performance.

II. RELATED WORK AND MOTIVATION

Database management systems have long used cost-based optimizers to select physical query plans. These optimizers are powerful but operate inside the engine and usually do not explain application-level query intent to analysts. Query monitoring systems and warehouse telemetry can identify slow or expensive workloads, but they often require platform-specific integrations and manual diagnosis.

LLMs have demonstrated strong performance on code understanding and transformation tasks. SQL is a promising target because it is structured, widely used, and often amenable to local validation. At the same time, SQL rewrites are high risk: equivalence depends on data semantics, null behavior, joins, aggregation, and engine-specific syntax. This motivates an architecture where LLM recommendations are recorded and validated rather than trusted directly.

For IEEE BigData, the relevant gap is not only query optimization but reproducible evaluation. Big data papers frequently rely on large systems and proprietary workloads that are difficult to reproduce. This work instead emphasizes an artifact-first benchmark: public datasets, scripted workload execution, persistent query history, and explicit cost accounting.

III. SYSTEM DESIGN

The framework is organized as six components. Each component maps to repository scripts to keep the implementation

DataSource → ExecutionEngine → QueryHistoryStore
 → LLMAnalyzer → RecommendationEngine
 → ReportGenerator

Fig. 1. Framework architecture for reproducible LLM-powered query monitoring.

TABLE I
QUERY HISTORY SCHEMA SUMMARY

Table	Purpose
workloads	Groups queries for an experiment run.
datasets	Tracks public dataset sources and local paths.
queries	Stores baseline and intentionally inefficient SQL.
query_executions	Captures execution status, runtime, rows, checksums, and errors.
llm_analyses	Stores model scores, explanations, issues, tokens, latency, and cost.
recommendations	Stores rewritten SQL and model rationale.
cost_comparisons	Links original and rewritten executions with semantic-match metadata.

auditable.

DataSource (`maintain_datasets.py`) downloads, validates, and stages public datasets. **ExecutionEngine** (`execute_query_workload.py`) runs baseline and intentionally inefficient SQL queries against DuckDB. **QueryHistoryStore** (`create_db.py`, `schema/query_history_schema.sql`) creates and maintains a SQLite database containing workloads, datasets, queries, executions, LLM analyses, recommendations, and before/after comparisons. **LLMAnalyzer** (`llm_analysis.py`) sends query context to the model and records structured output including scores, issues, token counts, latency, and cost. **RecommendationEngine**, implemented inside `llm_analysis.py`, extracts rewritten SQL and improvement rationale from model responses. **ReportGenerator** (`generate_report.py`, `cost_analysis_report.py`) computes accuracy, semantic-match, runtime, and cost summaries.

This decomposition separates data preparation, query execution, model analysis, validation, and reporting. The design also allows future work to replace DuckDB with production warehouses or replace the LLM provider without changing the persistent evaluation schema.

IV. IMPLEMENTATION

A. Query History Store

The query history schema is defined in `schema/query_history_schema.sql`. Table I summarizes the stored entities. The full schema is omitted from the main paper to preserve the IEEE page budget and is included in the artifact.

The schema links each recommendation to the original query, the model analysis that produced it, and the execution records used to validate it. This traceability is important

TABLE II
PILOT EVALUATION RESULTS

Metric	Result
Public datasets	3
Queries evaluated	8
Baseline queries	4
Intentionally inefficient queries	4
Detection accuracy	100%
False-positive rate	0%
Semantic-match rate for rewrites	75%
Total LLM API cost	\$0.000671

for big data governance because recommendations must be explainable and auditable.

B. LLM Analysis and Validation

For each query, the LLM analyzer constructs a prompt containing SQL text, workload metadata, expected issue labels, and execution context. The model returns structured JSON containing a score, explanation, detected issues, and optionally a rewritten query. The framework records token counts and estimated API cost to make the cost of model-assisted review explicit [5].

The validation path is deliberately conservative. A recommendation is re-executed through the same workload harness, and the result is compared to the original query using row counts and checksums where applicable. This does not prove full SQL equivalence, but it is sufficient to catch many unsafe rewrites in a reproducible pilot setting.

V. EXPERIMENTAL SETUP

The pilot workload uses three public datasets: USGS earthquake records [6], NOAA weather records [7], and the AWID intrusion-detection dataset [8]. These sources cover scientific, environmental, and cybersecurity data, matching the variety dimension of big data systems.

For this pilot, each table is reduced to three rows. This design choice isolates anti-pattern detection and rewrite validation from dataset-size effects. It also makes the artifact inexpensive to run and easy to inspect. The workload contains eight queries: four baseline queries and four intentionally inefficient variants. The inefficient variants target common SQL anti-patterns such as avoidable broad scans and unnecessary cross joins.

The evaluation measures four outcomes: detection accuracy on intentionally inefficient queries; false-positive rate on baseline queries; semantic-match rate for rewritten queries; and LLM API cost.

VI. RESULTS

Table II summarizes the pilot evaluation.

The framework detected all intentionally inefficient queries and did not flag the baseline queries. Three of four rewritten queries matched the original query semantics under the implemented validation checks. The total API cost was less than one-tenth of one cent for the pilot workload.

These results should be interpreted carefully. Because the tables contain only three rows, runtime deltas are not meaningful evidence of production speedup. The main result is operational: the framework can run an end-to-end query-governance loop, record model output, validate rewrites, and produce reproducible cost and correctness reports.

VII. DISCUSSION

The pilot demonstrates that LLMs can be useful as query-review assistants when embedded in an execution and validation harness. The framework does not assume the model is correct; it treats model output as a recommendation that must be stored, executed, and checked. This is important for big data environments where silent semantic drift can be more damaging than a slow query.

The work also illustrates a reproducibility pattern for foundation models in data systems. Rather than reporting only model responses, the artifact captures prompts, scores, token counts, cost, rewrites, execution outcomes, and validation status. This makes the evaluation inspectable and extensible.

VIII. LIMITATIONS AND FUTURE WORK

The current evaluation is intentionally small. It uses eight queries and 3-row tables, so it does not establish statistical confidence or production-scale runtime behavior. The execution engine currently targets DuckDB only; future work should add connectors for Snowflake, BigQuery, Redshift, PostgreSQL, and Spark SQL. SQLite is appropriate for a local artifact but not for high-concurrency telemetry. The validation checks compare row counts and checksums where possible, but they do not prove equivalence for all SQL constructs.

Future work should expand the query corpus, evaluate larger datasets, compare multiple LLMs, test prompt versions, add warehouse connectors, and study reviewer-facing explanations for query governance. A stronger BigData submission would also include larger-scale experiments before the final deadline.

IX. CONCLUSION

This paper presents a reproducible framework for LLM-powered SQL query monitoring and optimization. The framework integrates public datasets, DuckDB execution, SQLite query history, structured LLM analysis, rewrite validation, and cost reporting. In a pilot workload of eight queries across three datasets, it achieved 100% detection accuracy for tested anti-patterns, zero false positives, a 75% semantic-match rate for rewrites, and \$0.000671 in total LLM API cost. The results are not production-scale claims; they are evidence that foundation-model-assisted query governance can be evaluated transparently through an auditable artifact.

REFERENCES

- [1] M. Raasveldt and H. Muehleisen, "Duckdb: An embeddable analytical database," in *Proceedings of the 2020 Conference on Innovative Data Systems Research (CIDR)*, 2020. [Online]. Available: <https://www.cidrdb.org/cidr2020/papers/p29-raasveldt-cidr20.pdf>
- [2] DuckDB Foundation, "Duckdb documentation," Online documentation, 2026, accessed: 2026-07-12. [Online]. Available: <https://duckdb.org/docs/>
- [3] SQLite Consortium, "Sqlite documentation," Online documentation, 2026, accessed: 2026-07-12. [Online]. Available: <https://www.sqlite.org/docs.html>
- [4] S. Sharma, "Query monitoring framework," GitHub repository, 2026, accessed: 2026-07-12. [Online]. Available: <https://github.com/shouvik-sharma/query-monitoring-framework>
- [5] OpenAI, "Model documentation: Gpt-4o mini," Online documentation, 2026, accessed: 2026-07-12. [Online]. Available: <https://platform.openai.com/docs/models/gpt-4o-mini>
- [6] U.S. Geological Survey, "Earthquake hazards program: Earthquake catalog," Online dataset and documentation, 2026, accessed: 2026-07-12. [Online]. Available: <https://www.usgs.gov/programs/earthquake-hazards>
- [7] NOAA National Centers for Environmental Information, "Global hourly data," Online dataset and documentation, 2026, accessed: 2026-07-12. [Online]. Available: <https://www.ncei.noaa.gov/>
- [8] AWID Dataset Mirror, "Awid intrusion detection system 2020," GitHub repository, 2020, accessed: 2026-07-12. [Online]. Available: <https://github.com/krishanuskr/AWID-Intrusion-Detection-System-2020>